

Message Authentication Codes and Hash functions

Harshan Jagadeesh
Department of Electrical Engineering,
IIT Delhi



References and notes used from:

Cryptography and Network Security, Principles and Practices, by William Stallings
(combination of several editions)

Authentication Requirements

1. Disclosure
2. Traffic analysis
3. Masquerade
4. Content modification
5. Sequence modification
6. Source repudiation
7. Destination repudiation

Authentication functions

1. Message encryption

1. Ciphertext is itself the authenticator

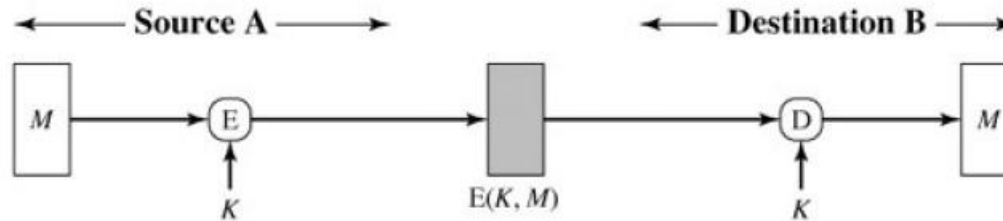
2. Message authentication code

1. A fixed tag is attached along with the message. The message can be in plaintext

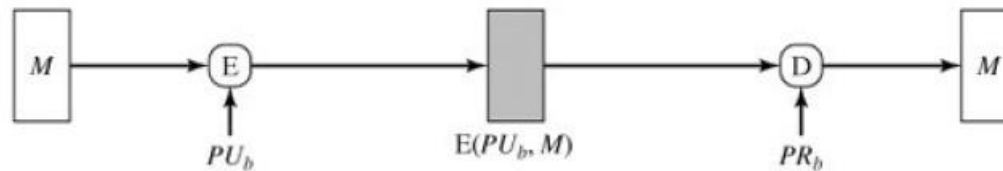
3. Hash function

1. A function that maps the message to a fixed-length value

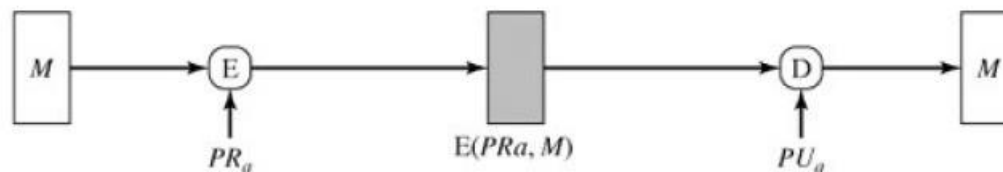
Message Encryption for Authentication



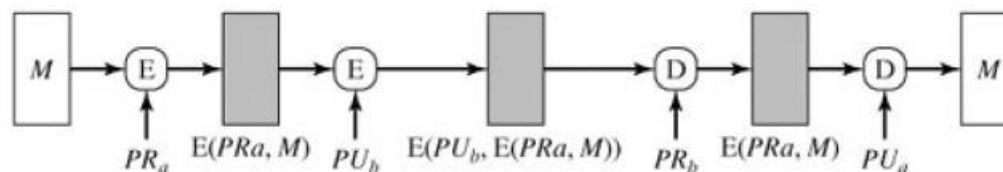
(a) Symmetric encryption: confidentiality and authentication



(b) Public-key encryption: confidentiality

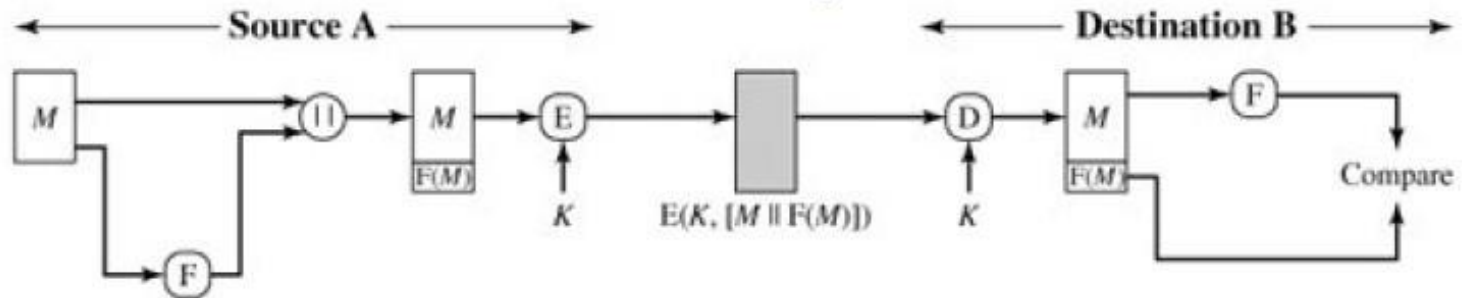


(c) Public-key encryption: authentication and signature

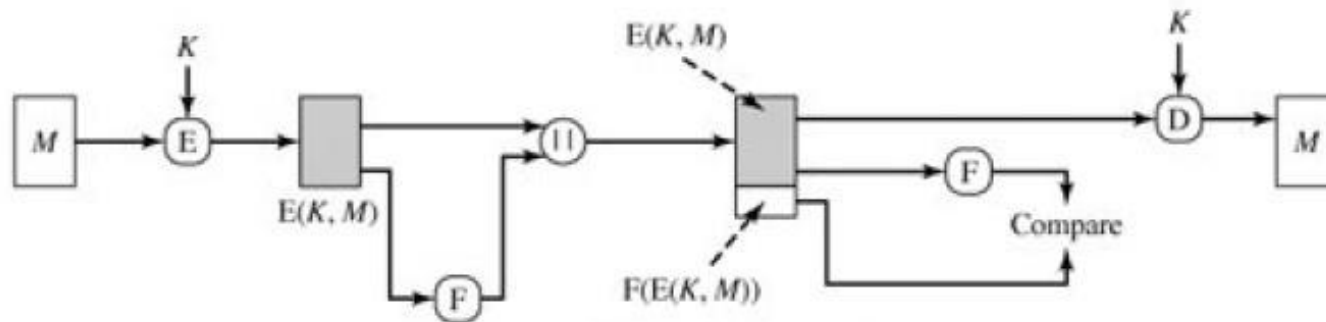


(d) Public-key encryption: confidentiality, authentication, and signature

Message Encryption for Authentication



(a) Internal error control



(b) External error control

Message Encryption for Authentication

$A \rightarrow B: E(K, M)$

- Provides confidentiality

Only A and B share K

- Provides a degree of authentication

Could come only from A

Has not been altered in transit

Requires some formatting/redundancy

- Does not provide signature

Receiver could forge message

Sender could deny message

Message Encryption for Authentication

$A \rightarrow B: E(PU_b, M)$

- Provides confidentiality
-

Only B has PR_b to decrypt

- Provides no authentication
-

Any party could use PU_b to encrypt message and claim to be A

Message Encryption for Authentication

$A \rightarrow B: E(PR_a, M)$

- Provides authentication and signature

Only A has PR_b to encrypt

Has not been altered in transit

Requires some formatting/redundancy

Any party can use PU_a to verify signature

$A \rightarrow B: E(PU_b, E(PR_a, M))$

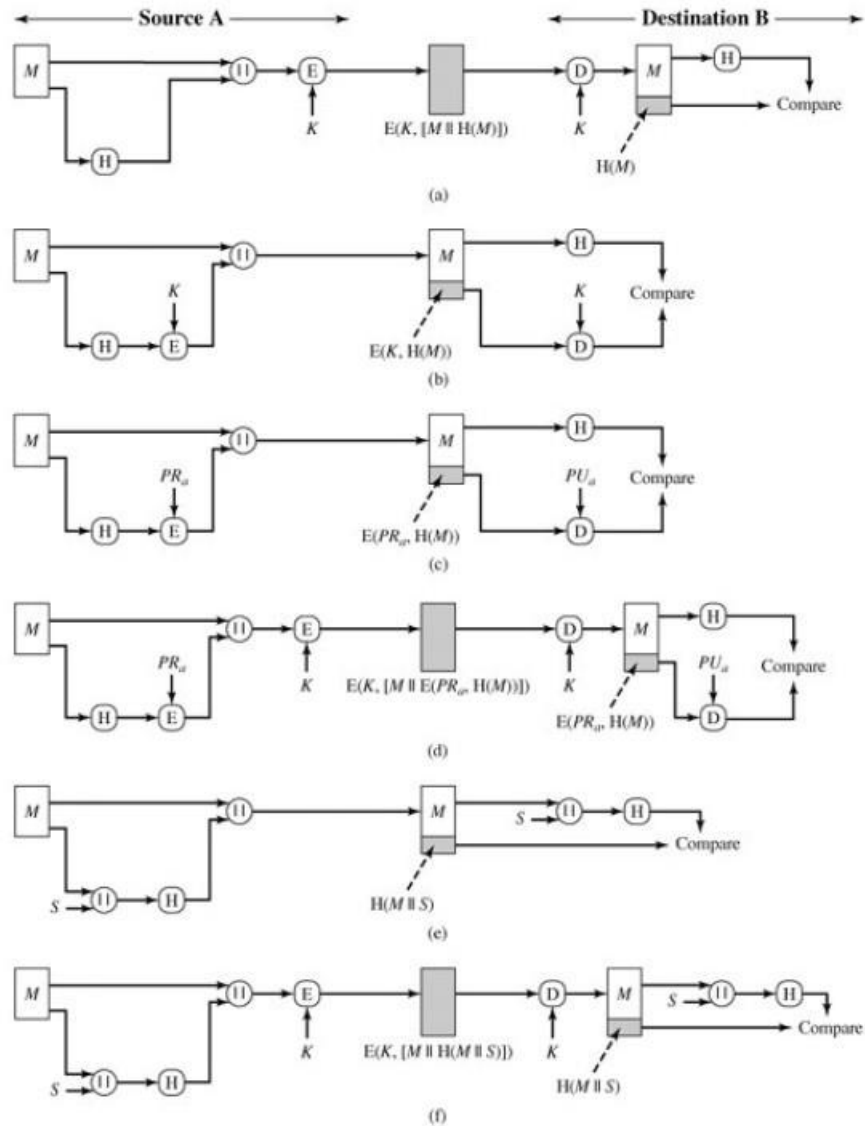
- Provides confidentiality because of PU_b
- Provides authentication and signature because of PR_a

(d) Public-key encryption: confidentiality, authentication, and signature

Message Authentication Code

$A \rightarrow B: M C(K, M)$
• Provides authentication
Only A and B share K
(a) Message authentication
$A \rightarrow B: E(K_2, [M C(K, M)])$
• Provides authentication
Only A and B share K_1
• Provides confidentiality
Only A and B share K_2
(b) Message authentication and confidentiality: authentication tied to plaintext
$A \rightarrow B: E(K_2, M) C(K_1, E(K_2, M))$
• Provides authentication
Using K_1
• Provides confidentiality
Using K_2
(c) Message authentication and confidentiality: authentication tied to ciphertext

Application of Hash Functions



Application of Hash Functions

$A \rightarrow B: E(K, [M H(M)])$	$A \rightarrow B: E(K, [M E(PR_a, H(M))])$
• Provides confidentiality	• Provides authentication and digital signature
Only A and B share K	• Provides confidentiality
• Provides authentication	Only A and B share K
$H(M)$ is cryptographically protected	
(a) Encrypt message plus hash code	(d) Encrypt result of (c) shared secret key
$A \rightarrow B: M E(K, H(M))$	$A \rightarrow B: M H(M S)$
• Provides authentication	• Provides authentication
$H(M)$ is cryptographically protected	Only A and B share S
(b) Encrypt hash code shared secret key	(e) Compute hash code of message plus secret value
$A \rightarrow B: M E(PR_a, H(M))$	$A \rightarrow B: E(K, [M H(M) S])$
• Provides authentication and digital signature	• Provides authentication
$H(M)$ is cryptographically protected	Only A and B share S
Only A could create $E(PR_a, H(M))$	• Provides confidentiality
	Only A and B share K
(c) Encrypt hash code sender's private key	(f) Encrypt result of (e)

Requirements for MAC?

1.

If an opponent observes M and $C(K, M)$, it should be computationally infeasible for the opponent to construct a message M' such that $C(K, M') = C(K, M)$.

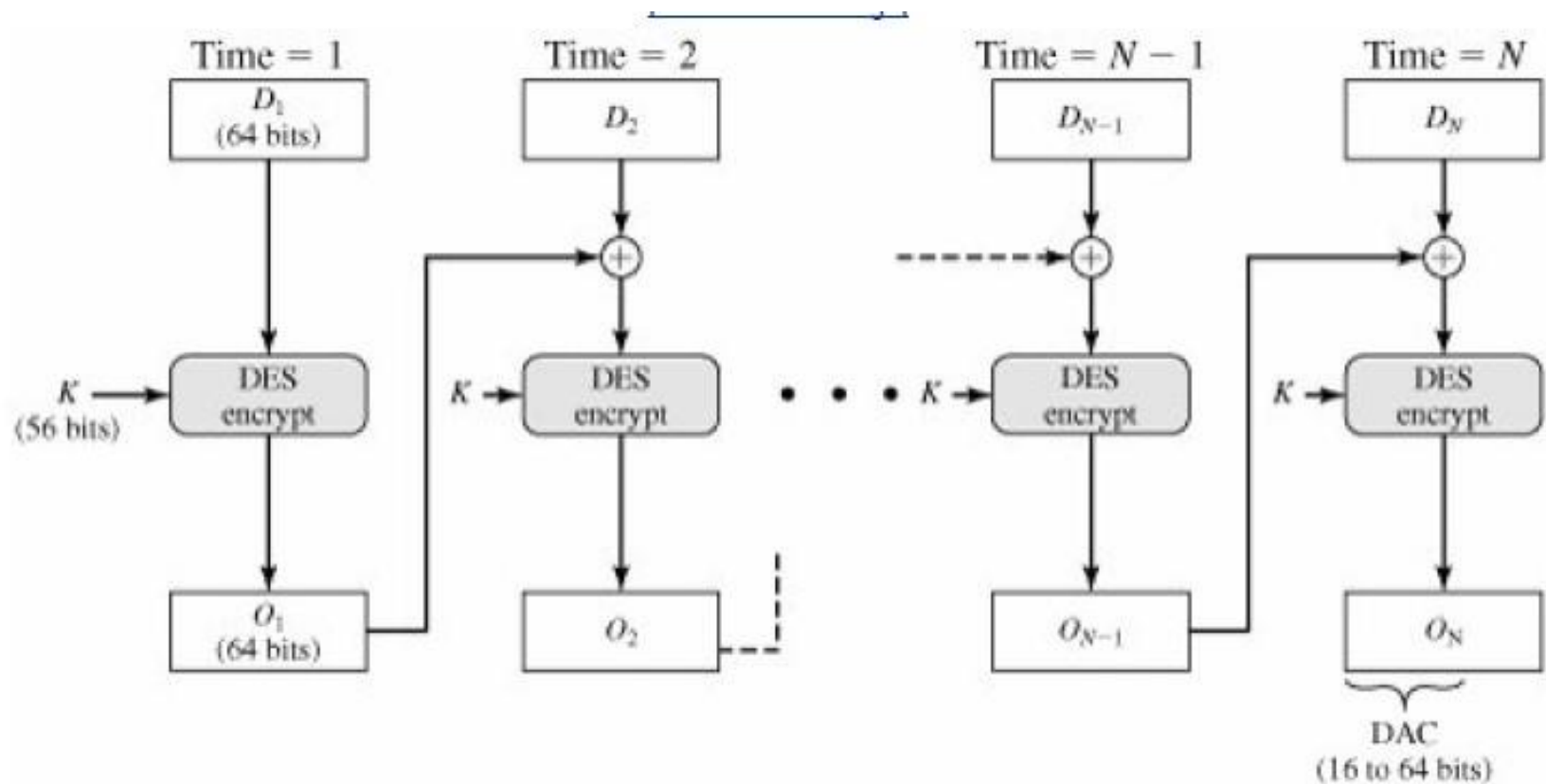
2.

$C(K, M)$ should be uniformly distributed in the sense that for randomly chosen messages, M and M' , the probability that $C(K, M) = C(K, M')$ is 2^{-n} , where n is the number of bits in the MAC.

3.

Let M' be equal to some known transformation on M . That is, $M' = f(M)$. For example, f may involve inverting one or more specific bits. In that case, $\Pr[C(K, M) = C(K, M')] = 2^{-n}$.

How to Design MAC?



Requirements on Hash Functions?

1.

H can be applied to a block of data of any size.

2.

H produces a fixed-length output.

3.

$H(x)$ is relatively easy to compute for any given x , making both hardware and software implementations practical.

4.

For any given value h , it is computationally infeasible to find x such that $H(x) = h$. This is sometimes referred to in the literature as the **one-way property**.

5.

For any given block x , it is computationally infeasible to find $y \neq x$ such that $H(y) = H(x)$. This is sometimes referred to as **weak collision resistance**.

6.

It is computationally infeasible to find any pair (x, y) such that $H(x) = H(y)$. This is sometimes referred to as **strong collision resistance**.^[2]

Implementation of Hash Functions?

$$C_i = b_{i1} \oplus b_{i1} \oplus \dots \oplus b_{im}$$

where

C_i = i th bit of the hash code, $1 \leq i \leq n$

m = number of n -bit blocks in the input

b_{ij} = i th bit in j th block

$\oplus$ = XOR operation

Implementation of Hash Functions?

1.

Initially set the n -bit hash value to zero.

2.

Process each successive n -bit block of data as follows:

a.

Rotate the current hash value to the left by one bit.

b.

XOR the block into the hash value.

Implementation of Hash Functions?

Be cautious when using encryption

$$C = X_{N+1} = X_1 \oplus X_2 \oplus \dots \oplus X_N$$

$$X_1 = IV \oplus D(K, Y_1)$$

$$X_i = Y_{i-1} \oplus D(K, Y_i)$$

$$X_{N+1} = Y_N \oplus D(K, Y_{N+1})$$

But X_{N+1} is the hash code:

$$X_{N+1} = X_1 \oplus X_2 \oplus \dots \oplus X_N$$

$$= [IV \oplus D(K, Y_1)] \oplus [Y_1 \oplus D(K, Y_2)] \oplus \dots \oplus [Y_N \oplus \dots \oplus D(K, Y_N)]$$

Implementation of Hash Functions?

Birthday attacks

1. The sender uses his message to generate a m -bit hash code, and then encrypts the hash code using his private key
2. The opponent prepares $2^{\{m/2\}}$ variations of the message. The opponent also prepared $2^{\{m/2\}}$ variations of the fraudulent message that must be substituted
3. The two sets of messages are compared to find a message pair that produces the same Hash code. The probability of success by birthday paradox is more than 0.5
4. The opponent offers the valid variation to the sender for signature. Then it replaces the message with the variation of the fraudulent message. The recipient cannot detect this attack.

Implementation of Hash Functions?

Hash Chaining Idea

1. Use encryption algorithm to generate the hash code
2. H_0 be the initial hash value
3. Divide the message M into N blocks
4. $H_i = E(M_i, H_{i-1})$
5. H_N is the hash code

No **shared-key** is used in this idea

Implementation of Hash Functions?

Meet in the middle attack

1. Generate $2^{\{m/2\}}$ random blocks, and for each block X , compute the hash code
2. Generate $2^{\{m/2\}}$ random blocks of Y , and then decrypt using the hash code
3. Based on the birthday paradox, with high probability there will be an X and Y such that they meet in the middle

Practical Implementations

Secure Hash Algorithms

Whirpool Hash structure

HMAC design